SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	*Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.*		
Student Name:	M. Deepika	**Reg. No.:**	192372296
Section / Batch:	2023	**Date:**	26/08/2026

Code of Conduct

I M. Deepika / 192372296 *(Name / Reg No)* *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: M.Deepika

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
	GRAND TOTAL:		**100 Marks**

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
   a) Clustering effect
   b) Memory usage
   c) Performance under high load factor
   d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
   a) Clustering issues
   b) Collision resolution efficiency
   c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
   a) Practical significance
   b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
   a) Space complexity (stack usage)
   b) Ease of implementation
   c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Compare Linear probing and seperate chaining for collision handling in hashing based on:

a) clustering effect

b) Memory usage.

c) performance under high load factor.

d) which method is more suitable for large scale applications and why?

factor	Linear probing	separate chaining.
clustering	suffers from primary clustering	less clustering.
Memory	Uses less memory	Needs extra for linked lists.
High load factor	performance decreases quickly	handles high load better.
Cache performance	Good	Low as to pointer traversal.

a) clustering:- Linear probing stores collided elements in consecutive locations, causing primary clustering. seperate chaining stores collided elements in a list at the same index, so clustering is less serious.

b) Memory Usage:- Linear probing is memory-efficient. because all elements are stored inside

the table. seperate chaining needs additi
nodes and poiness.

c) High load factors:- Linear probing becomes slowes as the table get full. seperate can han -dle higher load factoss because multiple elem -ents can exists is one bucket.

d) Large-scale applications:- Seperate chaining is generally bettes for large-scale applications bea -cause it handles collisions and high load factoss more effichively.

2) Compare quick sost and merge sost for sorting a large linked list of 10 million nodes. Analy -ze memory requirements, cache behavios, and time complexity. which algosithm is preferable for linked lists and why? how does the answes change for arrays?

factos	Quicksost	Merge sost
Average time	O(n log n)	O(n log n)
worst-case	O(n2)	O(n log n).
Linked-list suitability	Moderate	Excellent
Extra space	Recursion stack	Recusion stack
stability	Usually not Stable	Stable.

ked lists:-
linked lists do not support fast random access. merge sort only requires sequential traversal. It guarantees O(n log n) time

cache Behavior

Linked lists have poor cache locality because nodes may be scattered in memory.

-Arrays:- for Arrays, Quick sort is often preferred because it is in-place and has good cache locality.

3) Compare Quadratic probing and Double Hashing in terms of:

a) clustering issues

b) collision resolution efficiency.

c) suitability for dense hash tables.

factors	Quadratic probing	Double Hashing.
Clustering	Reduces primary clustering but may have secondary clustering	very low clustering
Collision resolution.	Good	Better.
Dense tables	Less effective as table becomes full	more effective.

complexity	Simplies	more Complex.

a) clustering :-
Quadratic probing avoids consecutive slot clustering but can suffer from secondary clustering.

b) collision Resolution :-
Double hashing generally provides better distribution and therefore resolves collisions more efficiently.

c) Dense Hash Table
for high load factors, Double Hashing is generally better because it distributes elements more evenly.

4) Best-case vs worst-case time complexity :-

parameter	Best-case O best	worst-case worst
Definition	minimum time take for most favorable input. Eg: Already sorted array for insertion sort O(n).	maximum time taken for most unfavorable input. Eg: Reverse sorted array for Quick sort with first element as pivot - O(n²)
a) pratical significance	1) shows lower bound of algorithm. 2) Used for optimistic analysis.	1) shows upper bound guarantee. 2) most important for critical system

	3) Rarely occurs in real random data 4) Important for algorithms that can be optimized for best-case	Real-time system banking, aviation 3) Helps worst-case time decide
5) Impact on algorithm selection.	If we know input is often nearly sorted, we can choose. Eg: Insertion sort is excellent for nearly sorted data O(n). and in hybrid sorts.	If worst-case is unacceptable, we avoid that algorithm even if average is good. Eg: Quick sort O(n²) worst case is risky, so in security-sensitive libraries.

2) code comparison Logic:-

Recursive: void qs(arr, l, r) { if (l<r) { p = partion (); qs (p+1, r) ; } }.

Iteractive:- stack = [(l, r)]; while (stack) { l, r = stack.pop(); p = partition (); if (p-1>l) push (p+1, r); }.

3) Recursive vs Iterative Quick sort:-

parameters	Recursive Quicksort	Iteractive Quicksort
a) space complexity.	space = O(logn) average for call stack, O(n) worst case for skewed partition. Each recursive call pushes stack frame with low, high, pivot.	space = O(logn) average O(n) worst case for explicit manual stack used to store sub-array bounds. But we can optimize stack to always push larger.

b) Ease of implementation	very easy and intuitive. Natural divide and conquer. code is 10-15 lines. Easy to understand: quicksort(left); quicksort(right);	partition and process smaller first, reducing worst to O(logn). Complex. Need to manually manage a stack data structure stack.push(low, high). Loop logic with while(stack not empty).
c) Risk of stack overflow.	High Risk. for large n.(e.g million nodes) and worst-case recursion depth O(n). system call stack with overflow and program crashes. python recursion limit is ~1000. C++ stack also limited to ~8MB.	Low Risk. Uses heap memory for explicit stack, which is much larger than call stack. we can control stack, safer for large-scale data. Can be made to handle lot elements without crashing.